

WAP to implement with Following operations.

a) create a linked list

b) Deletion of first element, specified element and last

element in the list. c) Display the contents of linked list

Pseudo
→ create linked list

```
struct node {
```

```
    int data
```

```
    node *next;
```

```
};
```

```
void deletefirst () {
```

```
    struct node *temp;
```

```
    if (head == null)
```

```
    {
```

```
        printf("list is empty"); // empty list
```

```
        return;
```

```
    temp = head;
```

```
    head = head->next;
```

```
    free(temp);
```

```
}
```

```
void deletelast () {
```

```
    struct node *temp, *prev;
```

```
    if (head == null)
```

```
    {
```

```
        printf("empty"); // list is empty
```

```
        return;
```

```
    }
```

```
    if (head->next == null)
```

```
    {
```

```
        printf("Deleted element")
```

```
        free(head);
```

```
        head = null;
```

```
        return;
```

```
    }
```

```
    temp = head;
```

```
    while (temp->next != null) {
```

```
        prev = temp;
```

```
        temp = temp->next;
```

```
}
```



```

    printf("Deleted element\n");
    prev->next = NULL;
    free(temp);
}

```

```

void deletespecific (int value) {
    if (head == NULL) {
        printf("Empty\n");
        return;
    }
    if (head->data == value) {
        head = head->next;
        printf("Deleted element\n");
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Element not found\n");
        return;
    }
    prev->next = temp->next;
    printf("Deleted element\n");
    free(temp);
}

```

code:

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

```



```
void createlist ( int n ) {
```

```
    struct node *newNode, *temp;
```

```
    int data, i;
```

```
    if (n <= 0) {
```

```
        printf ("Number of nodes should be greater than 0.\n");
```

```
        return;
```

```
    }
```

```
    for (i = 1; i <= n; i++) {
```

```
        newNode = (struct node*) malloc (sizeof (struct node));
```

```
        if (newNode == NULL) {
```

```
            printf ("memory allocation failed.\n");
```

```
            return;
```

```
        }
```

```
        printf ("Enter data for node %d", i);
```

```
        scanf ("%d", &data);
```

```
        newNode->data = data;
```

```
        newNode->next = NULL;
```

```
        if (head == NULL) {
```

```
            head = newNode;
```

```
        } else {
```

```
            temp->next = newNode;
```

```
        }
```

```
        temp = newNode;
```

```
    }
```

```
    printf ("\n linked list created successfully.\n");
```

```
void deletefirst () {
```

```
    struct node *temp; *prev
```

```
    if (head == NULL) {
```

```
        printf ("list is empty. nothing to delete.\n");
```

```
        return;
```

```
    }
```

```
    temp = head;
```

```
    head = head->next;
```

```
    printf ("Deleted element : %d\n", temp->data);
```

```
    free (temp); }
```



```

void deletelast () {
    struct node *temp, *prev;
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

```

```

    if (head->next == NULL) {
        printf("Deleted Element: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }

```

```

    temp = head;
    while (temp->next != NULL) {

```

```

        prev = temp;
        temp = temp->next;
    }

```

```

    printf("Deleted element: %d\n", temp->data);
    prev->next = NULL;
    free(temp);
}

```

```

void deletespecific (int value) {

```

```

    struct node *temp = head, *prev = NULL;
    if (head == NULL) {

```

```

        printf("List is empty. Nothing to delete.\n");
        return;
    }

```

```

    if (head->data == value) {

```

```

        temp = head;

```

```

        head = head->next;

```

```

        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

```

```

    while (temp != NULL && temp->data != value) {

```

```

        prev = temp;

```

```

        temp = temp->next;
    }

```



```
if (temp == NULL) {
```

```
    printf("Element %d not found in the list.\n", value);  
    return;
```

```
}
```

```
prev->next = temp->next;
```

```
printf("Deleted element: %d\n", temp->data);
```

```
free(temp);
```

```
}
```

```
Void displaylist() {
```

```
    struct node *temp = head;
```

```
    if (head == NULL) {
```

```
        printf("list is empty\n");
```

```
        return;
```

```
    }
```

```
    printf("Linked list: ");
```

```
    while (temp != NULL) {
```

```
        printf("%d -> ", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
int main() {
```

```
    int choice, n, value;
```

```
    while (1) {
```

```
        printf("\n-- singly linked list operations --\n 1. create  
linked list\n 2. delete first\n 3. delete specific\n 4. delete last  
5. display list\n 6. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1:
```

```
                printf("Enter number of nodes: ");
```

```
                scanf("%d", &n);
```

```
                createList(n);
```

```
                break;
```


case 2:

deleteFirst();

break;

case 3:

printf("Enter value to delete:");

scanf("%d", &value);

deleteSpecific(value);

break;

case 4:

deleteLast();

break;

case 5:

displayList();

break;

case 6:

printf("Exiting...\n");

exit(0);

default:

printf("Invalid choice. Try again.\n");

}

}

return 0;

}

output:

-- Singly Linked list Operations

1. create Linked list

2. delete First

3. delete specific

4. delete end

5. display list

6. Exit

Enter your choice: 1

Enter number of nodes: 5

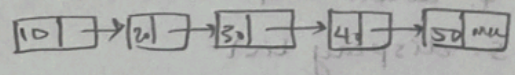
Enter data for node 1: 10

Enter data for node 2 : 20

Enter data for node 3 : 30

Enter data for node 4 : 40

Enter data for node 5 : 50

linked list created successfully. 

-- Singly linked list operations --

1. create linked list

2. delete first

3. delete specific

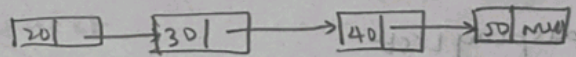
4. delete end

5. display list

6. Exit

Enter your choice : 2

deleted element : 10



-- Singly linked list operations --

1. create linked list

2. delete first

3. delete specific

4. delete end

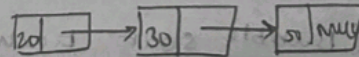
5. display list

6. Exit

Enter your choice : 3

Enter value to delete : 40

Deleted element : 40



-- Singly linked list operations --

1. create linked list

2. delete first

3. delete specific

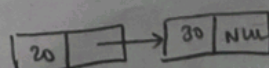
4. delete end

5. display list

6. Exit

Enter your choice : 4

Deleted element : 50



-- Singly linked list operations --

1. create linked list
2. delete first
3. delete specific
4. delete end
5. display list
6. Exit

Enter your choice : 5

Linked list : 20 → 30 → NULL

-- Singly Linked list operations --

1. create linked list
2. delete first
3. delete specific
4. delete end
5. display list
6. Exit

Enter your choice : 6

Exiting...